quindi nella prima versione di project.yaml consigli di integrare ambiente e file ambiente, porte, mount e host aggiuntivi, memoria e limiti risorse, come sono gestiti ora i mount aggiuntivi tra legacy e compose ? In python/bin/40.run.sh i volumi sono aggiunti alla variabile run_args, dovrebbero essere composti a partire da project.yaml ? Inoltre : sarebbe piu opportuno adottare il nome runtime.yaml anziche' project.yaml ?

Sì: i mount passati a `run_args` dovrebbero derivare da un **modello runtime comune**, ma non tutti devono essere dichiarati manualmente nel file del progetto.

Distinguerei:

- mount standard dell'ambiente Python, definiti dalla configurazione del linguaggio;
- mount aggiuntivi del progetto, dichiarati nel file runtime;
- mount dei servizi, definiti nei rispettivi manifest e template.

E, per il perimetro che stiamo assegnando a questo file, preferisco il nome `runtime.yaml` a `project.yaml`.

1. Come sono gestiti attualmente i mount

Nella versione allegata, lo script di esecuzione — ancora chiamato `30.run.sh`, da rinominare in `40.run.sh` secondo la decisione approvata — costruisce direttamente questi argomenti:

```
run_args=(
  --name "$CONTAINER_NAME"
  --hostname "$CONTAINER_NAME"
  -v "${PROJECT_DIR}:${APP_DIR_IN_CONTAINER}"
  -v "${VOLUME_NAME}:${VENV_DIR_IN_CONTAINER}"
  -v "uv-python:/home/${USER_}/.local/share/uv/python"
  "$IMAGE_NAME"
)
```

Quindi i tre mount standard sono definiti nello script Python, non in un modello dichiarativo comune. Gli eventuali mount aggiuntivi possono arrivare tramite `DOCKER_RUN_EXTRA_ARGS`. [1] [2]

La modalità Compose non è ancora implementata nei file allegati: non esiste quindi, al momento, una gestione comune già operativa.

2. Come dovrebbe funzionare dopo la modifica

La composizione proposta è:

```
mount standard del linguaggio
              +
mount aggiuntivi del progetto
              ↓
validazione e risoluzione comune
              ↓
lista normalizzata dei mount di dev
              ├─ legacy  → argomenti di docker run
              └─ Compose → services.dev.volumes
```

`40.run.sh` non deve più conoscere o ricostruire indipendentemente tutti i mount Python: deve consumare il risultato risolto.

Questo non significa eliminare `run_args`. L'array resta il contenitore finale degli argomenti nel backend legacy; cambia la loro provenienza.

Mount standard del linguaggio

La configurazione Python continua a fornire automaticamente:

Sorgente	Destinazione
Directory del progetto	`/app`
Volume virtualenv risolto	`/app/.venv`
Volume `uv-python`	`/home/<utente>/.local/share/uv/python`

Sono requisiti dell'ambiente attuale e devono essere preservati senza obbligare ogni progetto a ripeterli. [3] [1]

Mount aggiuntivi del progetto

Il progetto dichiara soltanto le aggiunte:

```
# <PROJECT_DIR>/.docker-dev/runtime.yaml
schema_version: 1

mounts:
  - type: bind
    source: ./datasets
    target: /datasets
    read_only: true

  - type: bind
```

```
    source: ./config
    target: /opt/project-config
    read_only: true
```

Il resolver interpreta `./datasets` e `./config` relativamente alla directory del progetto, non alla directory `.docker-dev` o alla directory corrente del comando.

La sintassi strutturata distingue il tipo di mount, la sorgente, la destinazione e l'accesso in sola lettura; Compose dispone delle corrispondenti proprietà nella sintassi estesa dei mount. [4]

3. Traduzione del medesimo mount

Per questa dichiarazione:

```
mounts:
  - type: bind
    source: ./datasets
    target: /datasets
    read_only: true
```

il modello risolto potrebbe contenere:

```
type: bind
source: /home/utente/REP/DEV/Python/foo/datasets
target: /datasets
read_only: true
```

Backend legacy

Il traduttore genera un array:

```
mount_args=(
  --mount \
  "type=bind,source=/home/utente/REP/DEV/Python/foo/datasets,target=/datasets,readon
)
```

Lo script di esecuzione lo incorpora prima dell'immagine:

```
run_args=(
  --name "$CONTAINER_NAME"
  --hostname "$CONTAINER_NAME"
  "${runtime_args[@]}"
  "$IMAGE_NAME"
)

docker run "${run_args[@]}" "${CONTAINER_CMD[@]}"
```

`runtime_args` comprende i mount standard e aggiuntivi, oltre alle altre impostazioni runtime supportate. Lo schema mostrato è illustrativo: il passaggio Perl → Bash deve conservare gli

argomenti, senza ricostruirli con `eval`.

Backend Compose

Il renderer genera:

```
services:
  dev:
    volumes:
      - type: bind
        source: /home/utente/REP/DEV/Python/foo/datasets
        target: /datasets
        read_only: true
```

La sintassi estesa rende esplicita la distinzione tra bind mount e volume nominato.[5] [4]

4. Volume nominato: mount e preparazione sono responsabilità distinte

Un progetto può anche richiedere un volume aggiuntivo:

```
schema_version: 1

mounts:
  - type: volume
    source: project-cache
    target: /cache
    read_only: false
```

Ma `project-cache` dovrebbe essere una chiave logica, non necessariamente il nome Docker definitivo.

Serve una dichiarazione separata della risorsa, per esempio:

```
volumes:
  project-cache:
    name: cache.dev.python.foo
    lifecycle: managed
```

Questi campi rappresentano una proposta di schema di `docker-dev`, non una sezione da passare direttamente a Compose.

La distinzione è:

Aspetto	Responsabilità
Nome, lifecycle e inizializzazione del volume	Preparazione delle risorse
Destinazione e accesso del mount	Configurazione runtime del container
Traduzione in `--mount` o YAML Compose	Backend

Per Compose, il mount usa la chiave logica e la dichiarazione top-level associa tale chiave al nome effettivo del volume. [5]

Il lifecycle deve avere un significato comune: non dovrebbe cambiare accidentalmente a seconda del backend selezionato. Ownership e inizializzazione dei volumi aggiuntivi restano un punto da formalizzare; non basta aggiungerli alla lista dei mount.

5. Regole di composizione da fissare

Non adotterei una regola indiscriminata “il progetto sovrascrive il linguaggio”.

Per la prima versione:

1. I mount standard vengono sempre inseriti.
2. I mount aggiuntivi vengono aggiunti.
3. Una destinazione duplicata produce errore.
4. I mount standard non sono sostituibili implicitamente.
5. Una sorgente bind mancante produce errore, salvo una politica di creazione esplicitamente dichiarata.
6. I mount dei servizi non vengono inseriti automaticamente nel container `dev`.

Per esempio, questa dichiarazione dovrebbe fallire:

```
mounts:
  - type: bind
    source: ./alternative-venv
    target: /app/.venv
```

Sostituirebbe il virtualenv gestito da Python. Un eventuale override richiede una funzionalità dedicata, non una conseguenza dell’ordine di merge.

Va invece consentita la combinazione standard:

```
bind del progetto → /app
volume virtualenv → /app/.venv
```

Quindi non bisogna vietare tutti i mount annidati: occorre proteggere le destinazioni gestite e valutare gli altri casi.

6. Perché `runtime.yaml` è un nome migliore

Preferisco:

```
<PROJECT_DIR>/.docker-dev/runtime.yaml
```

perché il file descrive come eseguire il container del progetto:

- ambiente e file ambiente;
- porte;
- mount;
- host aggiuntivi;
- memoria condivisa;
- limiti di risorse.

`project.yaml` suggerisce invece una configurazione più ampia: identità del progetto, build, linguaggio, servizi, alias e altre informazioni.

Con il nome `runtime.yaml`, eviterei anche una sezione `runtime:` ridondante:

```
schema_version: 1

environment:
  APP_MODE: development

env_files:
  - ./development.env

ports: []

mounts:
  - type: bind
    source: ./datasets
    target: /datasets
    read_only: true

extra_hosts:
  host.docker.internal: host-gateway

shm_size: 1g

resources:
  memory: 2g
  cpus: 2
```

La scelta comporta una delimitazione utile: alias dei servizi e configurazione della build non dovrebbero finire in questo file solo perché è disponibile.

7. Conseguenza sull'architettura della pipeline

C'è un ultimo punto importante: la risoluzione del runtime serve anche in legacy, quindi non può dipendere esclusivamente dall'attivazione dei servizi Compose.

Proporrei che il main invochi il componente Perl comune per risolvere il runtime prima della preparazione delle risorse:

```
main
  → risoluzione runtime comune
```

```
  → piano dei volumi e dei mount
  → 10.setup-volumes.sh
  → 20.build.sh
  → 30.services.sh
  → 40.run.sh
```

Se `runtime.yaml` introduce volumi aggiuntivi, il piano deve essere disponibile alla fase di setup; generarli soltanto dentro `40.run.sh` sarebbe troppo tardi per gestirne correttamente inizializzazione e ownership.

La proposta finale è quindi: adottare `runtime.yaml` per le esigenze aggiuntive del container di sviluppo, mantenere i mount standard nella configurazione del linguaggio e comporre entrambi in un modello comune. `run_args` e `services.dev.volumes` diventano due rappresentazioni dello stesso risultato, non due configurazioni indipendenti.

⁂

1. 30.run.sh
2. docker-run-extra-args.sh
3. entrypoint.Python.sh
4. https://timesofcloud.com/docker/compose/volume-syntax/
5. https://stackoverflow.com/questions/70798717/mount-volume-from-host-in-dockerfile-long-format
6. https://github.com/jbampton/docker-docs/blob/main/compose/compose-file/compose-file-v3.md
7. https://github.com/idontusenumbers/docker.github.io/blob/master/compose/compose-file/compose-file-v2.md
8. https://docs.docker.com/reference/compose-file/volumes/
9. https://docs.docker.com/reference/compose-file/
10. https://hostim.dev/learn/docker/docker-compose-volumes/
11. https://www.flowql.com/en/blog/guides/docker-compose-volumes/
12. https://forums.docker.com/t/mount-bind-long-vs-short-syntax/148628
13. https://github.com/Jaqueline-R/docker.github.io/blob/master/compose/compose-file/compose-file-v2.md